МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №5

по дисциплине Построение и анализ алгоритмов

Студент гр. 4343 _____ Пимахин Р.А

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Реализовать алгоритм Ахо-Корасик для множественного точного поиска набора образцов в тексте.

Вариант 4. Реализовать режим поиска, при котором все найденные образцы не пересекаются в строке поиска (т.е. некоторые вхождения не будут найдены; решение задачи неоднозначно).

Описание алгоритма

Алгоритм Ахо-Корасик

Основная идея: алгоритм Ахо-Корасик позволяет найти все вхождения множества строк-образцов в текст за линейное время O(|T| + Σ|Pi|). Алгоритм состоит из трёх этапов: построение бора, построение суффиксных и конечных ссылок, поиск в тексте.

Построение бора:

Бор (префиксное дерево) строится по всем образцам над алфавитом {A, C, G, T, N}. Каждая вершина хранит пять переходов (по числу символов алфавита). При добавлении образца символы переводятся в индексы 0..4, и для отсутствующих переходов создаются новые вершины. Вершина, в которой заканчивается образец, отмечается как терминальная с указанием его номера.

Построение автомата (суффиксные и конечные ссылки):

Для каждой вершины вычисляется суффиксная ссылка - вершина, соответствующая наибольшему собственному суффиксу строки текущей вершины; ссылки строятся обходом в ширину (BFS) от корня. Конечная ссылка (сжатая суффиксная ссылка) указывает на ближайшую терминальную вершину, достижимую по суффиксным ссылкам. Одновременно с этим недостающие переходы вершины достраиваются по переходам её суффиксной ссылки, поэтому итоговый автомат имеет определённый переход по любому символу из любой вершины.

Поиск в тексте:

Обход текста происходит по готовому автомату. На каждом шаге:

- выполняется переход по текущему символу в автомате;
- проверяется текущая вершина на наличие образца;
- проверяется цепочка конечных ссылок для нахождения всех образцов, оканчивающихся в текущей позиции.

Индивидуализация (вариант 4)

Удаление пересекающихся вхождений:

- все найденные автоматом вхождения собираются в виде троек (начало, конец, номер образца);
- тройки сортируются по возрастанию начала, а при равных началах - по возрастанию конца (то есть длины образца);
- вхождения перебираются по порядку: очередное добавляется в ответ, только если его начало больше конца последнего уже добавленного;
- поскольку сортировка ведётся по началу, а не по концу интервала, при равных началах в ответ попадает более короткий образец — выбор конкретного непересекающегося подмножества не единственен, что соответствует условию варианта 4.

Оценка сложности:

Временная сложность:

- построение бора: $O(\Sigma|Pi|)$, $\Sigma|Pi|$ - сумма длин образцов;
- построение суффиксных и конечных ссылок: $O(\Sigma|Pi| * |\Sigma|)$;
- поиск в тексте: $O(|T| + Z)$, где Z - число найденных (до фильтрации) вхождений;
- удаление пересечений: $O(Z \log Z)$ на сортировку и $O(Z)$ на линейный проход.

Общая сложность: $O(\Sigma|Pi| * |\Sigma| + |T| + Z \log Z)$, где в худшем случае (например, текст из одного повторяющегося символа) $Z = O(|T| * n)$.

Сложность по памяти: O(Σ|Pi| * |Σ| + T), где |Σ| = 5 - размер алфавита, Σ|Pi| - сумма длин образцов.

Структура данных для хранения автомата: для каждой вершины используется массив переходов фиксированного размера 5. Такой выбор обусловлен малым размером алфавита и даёт доступ к переходу за O(1).

Тестирование:

Входные данные	Вывод
ACGTAACGT 3 ACG GTA CGT	1 1 6 1
ABABAB 1 AB	1 1 3 1 5 1
AAA 1 A	1 1 2 1 3 1
ABBAABBA 2 AB BA	1 1 3 2 5 1 7 2
ACBBCCB 2 ACB BB	1 1

Вывод

В ходе лабораторной работы реализован алгоритм Ахо-Корасик для множественного поиска образцов в тексте. Алгоритм строит бор по образцам, достраивает его до конечного автомата с помощью

суффиксных и конечных ссылок и находит все вхождения образцов за один проход по тексту.

В рамках индивидуализации по варианту 4 реализован поиск, выдающий только попарно непересекающиеся вхождения: найденные автоматом совпадения сортируются по позиции начала и обрабатываются жадным алгоритмом выбора интервалов.